

SQL & DBMS Notes for Yardi Interview

These notes summarize core SQL and DBMS topics frequently asked in Yardi interviews, with examples based on a rental property schema (property, unit, tenant, lease, payment). [file:22]

1. Keys, Constraints, Normalization, ACID

1.1 Keys and Constraints

- **Primary key:** Uniquely identifies a row and cannot be NULL.
Example: `property_id` , `unit_id` , `tenant_id` , `lease_id` , `payment_id` . [web:24][file:22]
- **Foreign key:** References a primary key in another table to enforce relationships.
Example:
 - `unit.property_id` → `property.property_id`
 - `lease.unit_id` → `unit.unit_id`
 - `lease.tenant_id` → `tenant.tenant_id`
 - `payment.lease_id` → `lease.lease_id` [web:24][file:22]
- **NOT NULL:** Makes a column mandatory.
Example: `property.name` , `tenant.full_name` should be NOT NULL. [web:24][file:22]
- **UNIQUE:** Ensures all non-NULL values are distinct.
Example: UNIQUE on `tenant.email` to avoid duplicate emails. [web:24][file:22]
- **CHECK:** Restricts values using a condition.
Example: CHECK on `lease.status` to allow only 'Active' or 'Ended' . [web:24][file:22]

1.2 Normalization (1NF, 2NF, 3NF)

- **Normalization:** Organizing tables and relationships to reduce redundancy and avoid update anomalies. [web:24][web:27]

Normal form	Definition	Example (rental domain)
1NF	No repeating groups; every	Instead of storing <code>unit_numbers = 'A-101,A-102'</code> in one cell, move units into separate rows in a <code>unit</code> table. [web:24][web:42]

Normal form	Definition	Example (rental domain)
	column is atomic.	
2NF	1NF + non-key attributes depend on the whole composite key.	If <code>lease_detail(lease_id, month, amount, tenant_name)</code> exists, move <code>tenant_name</code> into <code>tenant</code> because it does not depend on the full <code>(lease_id, month)</code> key. [web:24][web:25]
3NF	2NF + no non-key attribute depends on another non-key attribute.	If <code>city</code> determines <code>state</code> , avoid redundant <code>state</code> stored with every row; use a reference/location table if needed. [web:24][web:25]

- **Pros:** Less duplication, fewer anomalies, consistent data, cleaner design. [web:24][web:26]
- **Cons:** More tables, more joins; sometimes partially denormalized for reporting performance. [web:27][web:39]

1.3 ACID Properties

- **Atomicity:** Transaction is all-or-nothing.
Example: Insert lease + first payment together; if payment insert fails, rollback the lease. [web:24][file:22]
- **Consistency:** Transactions move DB from one valid state to another while respecting constraints.
Example: Foreign keys prevent a payment for a non-existent lease. [web:23][file:22]
- **Isolation:** Concurrent transactions do not corrupt each other's data.
Example: Two clerks recording payments do not see each other's partial changes. [web:24][web:23]
- **Durability:** Once committed, changes survive crashes.
Example: Committed rent payments remain after server restart. [web:23][file:22]

2. Joins, Subqueries, UNION, Nth Highest

2.1 Join Types

Join type	What it returns	Example (rental schema)
INNER JOIN	Only matching rows from both tables.	Tenants who have at least one lease: <code>join tenant and lease on tenant_id</code> . [web:31][file:22]
LEFT JOIN	All left rows + matching right rows; unmatched right columns are NULL.	All units and their leases if present so vacant units still show. [web:31][web:40]
RIGHT JOIN	All right rows + matching left rows; unmatched left columns are NULL.	Less common; could show all leases even if some units are missing (bad data). [web:31]
FULL JOIN	All rows from both sides, with NULLs where there is no match.	All tenants and all leases, including tenants without leases and leases without tenants. [web:34]

Use your own tables in explanations to show understanding of relationships. [file:22]

2.2 Subqueries

- **Subquery:** Query nested inside another query, used in `SELECT` , `FROM` , or `WHERE/HAVING` .
[web:24][web:28]

Common uses:

- Filter in `WHERE` using `IN` / `EXISTS` .
Example (concept): “Tenants who have **not** paid for Feb 2025” → outer query on `lease / tenant` with `NOT EXISTS` subquery on `payment` for that period. [web:28][file:22]
- Derived table in `FROM`.
Example (concept): Aggregate payments by lease and period in a subquery, then join to `unit / property` to show property-wise collection. [web:37][file:22]

2.3 UNION vs UNION ALL

Operator	Behavior	Conditions	When to use
UNION	Combines results of two queries and removes duplicates (does DISTINCT).	Same number of columns, same order, and compatible data types in both queries. [web:31][web:37]	When duplicates are not needed, e.g., unique list of tenants from two tables.
UNION ALL	Combines results and keeps duplicates; faster (no DISTINCT).	Same structural rules as UNION. [web:37]	When duplicates are acceptable or required, e.g., merging payment logs. [web:31]

Mention performance: UNION ALL is usually faster than UNION. [web:31][web:37]

2.4 Nth Highest Salary / Rent (Concept)

- Use a **window function** like `DENSE_RANK()` over `ORDER BY monthly_rent DESC`, then pick rows where rank = N (e.g., 3 for third highest). [web:24][web:37]
- Alternate conceptual method: repeatedly take `MAX(salary)` of rows strictly less than the previous max using nested subqueries. [web:24][web:33]

3. Indexes, Views, and Performance

3.1 Indexes

- **Index:** Separate structure (often B-tree) that stores key values + pointers to rows, accelerating lookups, joins, and filters at the cost of extra storage and slower writes. [web:29][web:41]

Where to index in rental schema:

Table	Column(s)	Reason
unit	property_id (FK)	Fast joins from property → units and efficient deletes/updates on property. [web:29][file:22]

Table	Column(s)	Reason
lease	unit_id , tenant_id (FKs)	Faster lease lookups by unit/tenant and better referential checks. [web:29]
payment	lease_id (FK)	Quick access to payments per lease. [web:29]
payment	(period_year, period_month)	Faster monthly/period-wise rent reports. [web:29]

Common interview point: indexing foreign keys is a best practice in OLTP systems. [web:29][web:41]

3.2 Views

- **View:** Virtual table defined by a SELECT; stores no data itself but reads from base tables. [web:24]

Uses:

- Simplify complex joins.
- Provide limited, secure access to underlying tables. [web:24]

Examples:

- vw_active_leases : active leases joined with tenant, unit, and property details. [file:22]
- vw_rent_collection_2025 : total payments per property for 2025. [file:22]

4. Procedures, Functions, Triggers, Cursors

4.1 Stored Procedure vs Function

Feature	Stored Procedure	Function
Purpose	Perform actions (DML, business logic).	Compute and return a value. [web:32]
Return	Optional; can use OUT parameters.	Mandatory single return value (scalar/record/collection). [web:32]

Feature	Stored Procedure	Function
Use in SQL	Generally not used directly in SELECT.	Can be used inside SELECT/WHERE if side-effect-free. [web:37]
Example	<code>create_lease</code> that inserts a lease and updates unit status to <code>'Occupied'</code> .	<code>calc_late_fee(lease_id)</code> returning numeric fee. [file:22]

Key line for interview: “Procedures are action-oriented; functions are value-oriented and used in expressions.” [web:32][web:37]

4.2 Triggers

- **Trigger:** Stored program that runs automatically on events like INSERT, UPDATE, or DELETE. [web:29][web:38]

Example uses in rental system:

- BEFORE INSERT on `payment` :
 - Derive `period_month` , `period_year` from `payment_date` if NULL.
 - Validate `amount > 0` . [file:22]
- AFTER UPDATE on `lease.status` :
 - When lease becomes `'Active'` , set `unit.status = 'Occupied'` .
 - When last active lease ends for a unit, set `unit.status = 'Vacant'` . [file:22]

Note: too many or too complex triggers can make debugging harder, so core logic is often kept in procedures when possible. [web:29][web:38]

4.3 Cursors

- **Cursor:** Mechanism in PL/SQL to process result rows one by one. [web:24]

Types:

- Implicit cursor: created automatically for each SQL statement.
- Explicit cursor: declared by developer (`CURSOR c IS SELECT ...`), then `OPEN` , `FETCH` , `CLOSE` . [web:24]

Use cases:

- Complex row-by-row transformations not easily done in pure SQL.

- Data migration or generating detailed, procedural reports. [web:33]

Drawback: slower than set-based operations; use only when needed. [web:24][web:33]

5. CRUD, DELETE/TRUNCATE/DROP, Dates, Strings

5.1 CRUD

CRUD corresponds to basic SQL operations. [web:24][file:22]

Operation	SQL	Example
Create	INSERT	Add a new tenant or lease .
Read	SELECT	Get all payment rows for a given lease_id .
Update	UPDATE	Change unit.monthly_rent or update tenant.phone .
Delete	DELETE	Remove a test payment row.

5.2 DELETE vs TRUNCATE vs DROP

Command	What it does	Transaction behavior	Structure
DELETE	Remove selected rows (with WHERE) or all rows (without WHERE). [web:24]	Row-by-row, fully logged; can be rolled back. [web:24]	Table and indexes remain.
TRUNCATE	Remove all rows quickly; no WHERE allowed. [web:24]	DDL-like, minimal logging; not normally rolled back after commit. [web:24]	Table and indexes remain but data gone.
DROP	Remove entire object (table, view, etc.). [web:33]	Once committed, cannot use the object unless recreated. [web:33]	Structure and data removed.

Usage:

- Use DELETE for business-driven record deletion.
- Use TRUNCATE for clearing temp/staging tables.

- Use DROP when a table is no longer needed. [web:24][web:33]

5.3 Dates in Oracle

- SYSDATE : Current database server date and time. [web:24][file:22]
- CURRENT_DATE : Current date/time in session time zone. [web:24]
- SYSTIMESTAMP : Timestamp with fractional seconds and time zone. [web:27]

Dates are critical for due dates, payment dates, and aging reports in rent systems. [file:22]

5.4 String Functions

Common string functions in SQL: [web:24][file:22]

- CONCAT / || – join strings, e.g., build full tenant name.
- SUBSTR – extract substring, e.g., block code from unit_number .
- LENGTH – string length, e.g., validate phone number length.
- UPPER / LOWER / INITCAP – change case for case-insensitive search.
- TRIM , LTRIM , RTRIM – remove spaces/characters.
- REPLACE – replace substring, e.g., update dummy email domains.

Tip for interview: for each topic, answer in this order – one-line definition → why it is needed → short example from your rental schema → any trade-off or special note. [file:22][web:52]